The information condensed by the prompt falls into two categories: (1) **task-dependent prompt tuning**, and (2) **instance-dependent prompt tuning**. Each category encompasses general and specific enhancements for domain and task adaptation. Although some studies are based on PLMs, the advantages apply to LLMs, given the correlation between prompt tuning enhancements and model size [72] and successful implementations on large-scale PLMs. Moreover, it provides a parameter-efficient, fully controllable tuning method to adapt PLMs for more customized purposes.

4.2.1 Task-dependent Prompt Tuning. Task-dependent prompt tuning optimizes a shared prompt for all instances within a specific task, enabling it to encapsulate information from extensive datasets comprising thousands or millions of examples. However, training a naïve prompt is hard to converge and suboptimal for different scenarios, leaving room for improvement for specific tasks and domains.

Prompt Content Enhancement. We refer prompt content as the embedding values of continuous prompt, enhancements are developed in terms of task-specific initialization and prior knowledge transfer. Pilot works have validated that in contrast to many optimizers that begin with a random distribution applied in general ML tasks, the optimization process of soft prompt is significantly influenced by its initial value. For language models, word embeddings are pre-trained to be quite distinct. Consequently, a standard optimizer such as stochastic gradient descent (SGD) can only update the parameters in a limited vicinity, leading to the possibility of falling into a local minimum [1]. Therefore, a more effective initialization approach would involve using embeddings of concrete task-specific words.

One of the pioneering works, WARP [43] initializes the prompt by the embedding of special token "[MASK]". KnowPrompt [18] designed learnable prompts as virtual type words and virtual answer words, which are initialized by the aggregated representation of concrete label words and disassembling words based on their frequency in the dataset. In addition, random initialization has been proven to be the least efficient, especially for small model, while Prompt-tuning [72] presented no significant gap between initialization strategies when the model size grows to 11B, indicating that LLMs is robust for prompt's initialization values in general tasks.

Further studies have revealed that retrained prompts on source domains can enhance performance in unseen target domains, illustrating the ability of prompt transfer [153]. SPoT [153] initialize the prompt with a single generic source prompt learnt from multiple sources tasks, and then fine-tune it on target task in a classic way [72]. PPT [40] also pre-trains a prompt using self-supervised learning on extensive unlabeled corpora, which then serves as the initial prompt for the target task. Su et al. [145] demonstrated the transferability of continuous prompts in both cross-task as well as cross-model settings, and find that a well-initialized prompt can significantly accelerate training convergence. Furthermore, take the advantage of transferability, LFPT5 [121] employed soft prompt for lifelong learning. It continuously trains the prompt that simultaneously learns to solve the current task and generate training samples of previous tasks to overcome the *catastrophic forgetting*. Progressive prompts [128] introduce the prompt tuning into continuous learning. The prompt for current task is defined as the concatenation of prompts that are optimized on previous tasks and a tunable current prompt.

Prompt Construction Enhancement. We refer to the prompt construction about the positioning and length of the prompt, and combinations of additional templates or discrete prompts. Continuous prompts can be simply prepended, appended, and inserted to the original input sentences without extra language phrases. The pioneering study, WARP [43] adopted all three intersections with a "[MASK]" token for classification tasks. In Prefix-tuning [79], tunable prompts are prepended to the sentence embedding and the activation of all attention blocks, capitalizing on the left-to-right

nature of the autoregressive model: the prepended prompt can efficiently affect the subsequent words through attention. In addition, a recent work [72] prepend prompts at the input layer, achieving comparable results to fine-tuned models.

Template is widely used to leverage the adaptation performance [136], for example, reformulating an NLP task (e.g., sentence classification) into the masked words prediction task that is employed during LM pre-training. Based on the predefined task-specific templates, soft prompts can be inserted and thus offers flexibility for conditional tuning. KnowPrompt [18] designed a template appending to the input sentence with a "[MASK]" between subject and object for relation extraction and incorporates trainable prompts of "virtual type words" surrounding these two entities. Output embeddings of "virtual type words" are trained to align logically with the target relation at the masked position, conditioning the optimization with the information of entity type. KiPT [77] developed a knowledge extractor for event detection tasks, which identifies trigger words in sentences based on their semantic similarity to event concepts above a threshold. Identified trigger words as well as the corresponding event labels will then be prepended to a randomly-initialized soft prompt with the input sentence. KiPT also reformulates the sequence tagging tasks: trigger-identification and trigger classification, into the generative task by outputting structured event records.

4.2.2 Instance-dependent prompt tuning. A shared task-dependent prompt is static against the change in input sentence, which ignores semantic differences as well as specific knowledge of individual instances, and thus be suboptimal in fine-grained objectives. Instance-dependent prompt tuning however conditionally generates prompts for individual instances, incorporating both contextual information and task instructions.

Prompt Content Enhancement. Enhancement of prompt content for instance-dependent tuning focus on learning a joint and adaptive representation of tasks as well as instance context. IDPG [170] proposed an additional two-layer perceptron as a prompt generator, down and up project the sentence embedding to the adaptive soft prompt. ATTEMPT [3] first train multiple prompts on large-scale source tasks, and calculate an aggregated prompt base on a sentence-wise attention network, which will then be mixed with a newly initialized target task prompt as the final instance-dependent prompt. Jin et al., [63] assume that prompt tokens differently contribute the instance, and thus designed a look-up module to score the association of prompt tokens to instance tokens, which is then used to calculate the aggregated prompt embeddings. Bhardwaj et al., [8] generate context-aware prompts by a transformer-based sentence encoder, but further quantize the contextual prompt into a more compact representation to avoid optimization collapse. Levine et al., [73] learn the joint representation of prompt and input by a frozen T5 encoder following cross- and self-attention layers. Liu et al., [85] propose an instance-aware prompt that is applied to the intermediate layers of LM. The proposed prompt generator is a simple feed-forward layer with bottleneck architecture which take the embedding of [CLS] token or pooling of embeddings of sentence tokens.

Prompt Construction Enhancement. Similar to the construction enhancement, instance-dependent prompt tuning introduce instance-dependent knowledge as concrete words or learn adaptive prompt in terms of positioning and length. OntoPrompt [177] enriches the template with instance-related knowledge from external ontology as an additional text, and tune continuous prompts surrounding the "[MASK]" to help prediction. Recently, to give a comprehensive discussion of the effect of content and structure of prompts, dynamic prompting [175] proposed a unified framework to learn an instance-dependent prompt by dynamically defining prompt position, length, and values for each instance. It also proves the effectiveness of post-fix prompt, given most prior works prepend the prompt to the input sentence.

4.2.3 Open Challenges. Continuous prompt tuning presents a streamlined method to utilize the broad language understanding capacity of LLMs for specific tasks across different domains. It efficiently tackles issues inherent in

discrete prompt methods, such as (1) significant reliance on the prompt for LLM performance, where minor wording or template changes can greatly affect the result, (2) computational complexity in identifying the optimal natural language-based prompt from a large search space, and (3) the time-consuming and labor-intensive process of manually designing instructions, particularly in expertise-required domains. However, continuous prompt tuning has its limitations.

(1) *Interpretability* is often criticized as a weakness of soft prompt tuning. By discreting the optimal continuous prompts into nearby token vectors in LM's vacabulary, studies such as WARP [43] have found these prompts to be non-interpretable and lacking meaningful content. In KnowPrompt [18] and Prompt-tuning [72], prompt tokens are discovered in close proximity to domain-related terms. For example, Prompts trained on the BoolQ dataset revealed that *science*, *technology*, and *engineering* were the nearest neighbors of the optimal prompt, as approximately 20% of the questions pertain to the "Nature/Science" category [72]. However, the interpretability of continuous prompts as a coherent sequence is still unclear. In addition, continuous prompt is not confined to directing LLMs using compact textual information. OPTIMA [41] achieves domain adapting with prompt by tuning it help regularizes the decision boundary to be smooth around regions where source and target data distributions are similar with an adversarial learning framework.

(2) *Limited access* to LLMs poses a significant challenge for continuous prompt learning, especially as models with immense sizes (e.g., 540B PaLM) and models with only API access. This restriction hinders differential optimization on continuous embeddings. In this case, derivative-free prompt tuning that optimizes the soft prompt without gradients from LLMs is widely discussed. Black-box tuning (BBT) [146] proposed a gradient-free approach to searching the optimal prompt in a smaller intrinsic space. with Covariance Matrix Adaptation Evolution Strategy (CMA-ES) for non-convex optimization instead of Adam. Similarly, Clip-Tuning [14] use multiple deterministic clipping instances of the target LM to optimize an agent that learns the intrinsic prompt embedding. However, those methods still need access at least to the embedding layer, which is unsatisfactory for LLMs where only textual query is allowed. In this case, derivative-free approaches for discrete prompt search appear to be a more promising direction, and several studies have already achieved preliminary success [29, 119].

5 MODEL FINE-TUNING FOR DOMAIN SPECIALIZATION

LLMs, despite being trained on extensive general text data, might not encode adequate knowledge for specific tasks or domains. In such scenarios, fine-tuning the model on a smaller, domain-specific dataset can enhance its performance within that particular area. This fine-tuning can be divided into two main approaches. Adapter-based Fine-tuning and Task-oriented Fine-tuning. (1) **Adapter-based Fine-tuning:** This approach, as illustrated in Figure 7 (a), employs neural adapters or modular components to enhance the LLM's performance on domain-specific tasks without major modifications to the LLM's inner parameters. These adapters, typically integrated into the existing LLM architecture, allow for task-specific learning while keeping the original model largely intact. (2) **Task-oriented Fine-tuning:** As represented in Figure 7 (b), this method focuses on modifying the LLM's inner

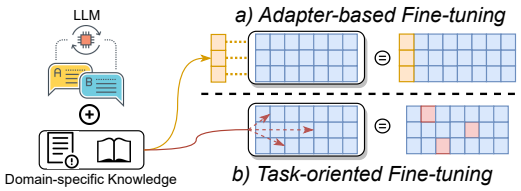

Fig. 7. The visualization of two approaches to fine-tune LLMs based on domain-specific knowledge, where the blue rectangle denotes the set of parameters in LLM. (a) the adapter-based LLM fine-tuning aims to fine-tune LLMs on specific domains with a small number of extra parameters (i.e., adapter); and (b) the task-oriented model fine-tuning aims to fine-tune LLMs based on specific tasks.